

2026年5月版

多 Agent 智能客服项目指南

面向面试与实战的企业级实现

聚焦金融/电商客服场景，覆盖 Supervisor 编排、RAG 检索、MCP 工具调用与可观测性，并提供 Python/Java/Go 三语言实现和面试资料

Focused on finance and e-commerce customer support, this guide covers supervisor orchestration, RAG retrieval, MCP tool calls, and observability, with Python/Java/Go implementations and interview materials

文档版本: v1.0.1

发布时间: 2026-05-20

语言: 简体中文

「小番薯生活集」官方整理，认准本店，持续更新！

本产品包含的产品和工具信息，功能描述可能随时变化，请以各产品工具官方文档为准。

目录

Table of Contents

- 01 项目概览
- 02 架构设计文档
- 03 代码讲解文档 — 核心模块逐行解析
- 04 部署指南
- 05 八股文题库 — 多Agent系统 & RAG & 大模型应用
- 06 项目深度问答 — 面试官追问 & 标准回答
- 07 简历项目经历模板 — 智能客服多Agent系统
- 08 STAR面试话术 — 智能客服多Agent系统

项目概览

企业级面试项目全攻略 — 面向金融/电商场景，包含 Python / Java / Go 三语言完整实现 + 配套面试材料，从零到面试一站搞定。



目录

- 项目简介
- 系统架构
- 核心功能
- 技术栈
- 三语言实现对比
- 快速开始
- 项目结构
- 核心代码解析
- 面试准备材料
- 参考项目
- 安全说明



项目简介

本项目是一个**企业级多Agent智能客服系统**，模拟真实金融/电商公司的客服场景。系统由多个专业AI Agent协同工作，自动处理用户咨询、工单创建、知识检索等任务。

这个项目能帮你做什么？

- ☒ **面试加分项**：拥有一个真实完整的多Agent项目，不再只是CRUD
- ☒ **三语言实现**：Python/Java/Go均有完整代码，适配不同岗位需求
- ☒ **面试材料齐全**：简历模板、STAR话术、八股文题库一应俱全

-  **学习参考：**代码有详细注释，架构文档有图文说明

适合人群：

- 准备AI/后端岗位面试的同学
- 想了解多Agent系统架构的开发者
- 对LangGraph/Spring AI/Go Agent开发感兴趣的工程师

系统架构

整体架构图

请求处理流程

核心功能

1. Supervisor 编排模式

什么是Supervisor？ 就像一个项目经理，接到需求后分配给不同专家处理，最后汇总结果。

特性	说明
中央协调	由Supervisor统一调度，子Agent只做专业工作
顺序编排	子Agent按意图路由 → 业务处理 → 合规审查顺序执行
断点恢复	使用LangGraph Checkpoint（Python），对话可中中断续接

2. 分层记忆系统

为什么需要三层记忆？ 类似人类记忆： 工作桌(工作记忆) + 笔记本(短期) + 大脑长期记忆。

记忆层	存储位置	生命周期	延迟	用途
工作记忆	进程内存 (dict/struct)	单次请求	<1ms	当前推理状态、路由决策上下文
短期记忆	Redis	TTL 30 分钟	1–5ms	多轮对话上下文 (保留最近20轮)
长期记忆	FAISS (Python) / 64维哈希+余弦相似度 (Java/Go)	永久	10–50ms	知识库、用户画像、历史工单

3. MCP 工具协议

什么是MCP？ Model Context Protocol，AI模型调用外部工具的标准协议，类似HTTP规范了Web通信。

```
json
{
  "name": "order_query",
  "description": "查询订单信息",
  "inputSchema": {
    "type": "object",
    "properties": {
      "order_id": {"type": "string", "description": "订单ID"}
    },
    "required": ["order_id"]
  }
}
```

已实现的MCP工具：

- `order_query` — 查询订单状态、物流信息
- `knowledge_search` — 知识库全文搜索
- `ticket_create` — 工单创建
- `risk_check` — 金融风控接口

4. RAG 知识检索

什么是RAG? Retrieval-Augmented Generation, 先从知识库检索相关内容, 再让AI生成回答, 避免AI"瞎编"。

5. 全链路追踪 (OpenTelemetry)

可以清楚地看到每次请求经过哪些Agent、每个步骤耗时多少、消耗了多少Token:

6. 合规审查

专为金融场景设计:

- **敏感词检测**: 自动识别违规词汇
 - **PII保护**: 过滤身份证、银行卡等隐私信息
 - **越权访问治理**: 防止用户绕过权限限制
 - **双重审查**: 规则引擎 (快, <2ms) + LLM审查 (准, ~600ms)
-

层次	技术选型	说明
AI框架	LangGraph (Python) / Spring AI (Java) / Gin+原生 HTTP (Go)	多Agent编排
LLM	GPT-4o / Claude 3.5	大语言模型
向量数据库	FAISS (开发) / Milvus (生产)	知识检索
缓存	Redis	短期记忆、会话管理
追踪	OpenTelemetry + Jaeger	全链路追踪
API	FastAPI / Spring Boot / Gin	REST接口
容器	Docker + Docker Compose	一键部署
协议	MCP (Model Context Protocol)	工具调用标准

三语言实现对比

维度	Python (LangGraph)	Java (Spring AI)	Go (Gin)
目录	<code>python-impl/</code>	<code>java-impl/</code>	<code>go-impl/</code>
编排框架	LangGraph StateGraph	Spring AI ChatClient + 手动编排	手动编排 + goroutine
HTTP框架	FastAPI	Spring Boot	Gin
LLM调用	LangChain ChatOpenAI	Spring AI ChatClient	自研OpenAI兼容HTTP 客户端
状态管理	TypedDict + Checkpoint	POJO	struct + sync.RWMutex
并行方式	asyncio	同步顺序调用	同步顺序调用
向量检索	FAISS (1536维)	64维哈希+余弦相似度	64维哈希+余弦相似度
合规审查	规则+LLM双阶段	规则+LLM双阶段 + PII 脱敏	规则+LLM双阶段 + PII 脱敏
生态	LangSmith / LangServe	Spring 全家桶	Gin + go-redis + 标准 库
适合场景	AI原型、数据科学团队	企业级金融/银行	高并发云原生微服务
生产成熟度	★★★★★	★★★★★	★★★
面试亮点	最主流AI开发栈	大厂Java面试必考	字节/腾讯Go岗位

快速开始

前置条件

- 一个 OpenAI API Key（或其他LLM的Key）
- Docker（可选，用于一键启动）

方式一：Docker 一键启动（推荐新手）

```
bash

# 1. 打开项目
cd code/

# 2. 配置API Key
cp python-impl/.env.example python-impl/.env
# 编辑 .env 文件，填入你的 OPENAI_API_KEY

# 3. 一键启动所有服务
docker-compose up -d

# 4. 访问接口
# API文档: http://localhost:8000/docs
# 追踪UI: http://localhost:16686 (Jaeger)
```

方式二：Python 版本（直接运行）

```
bash

# 进入Python实现目录
cd python-impl

# 安装依赖
pip install -r requirements.txt
```

```
# 配置环境变量
cp .env.example .env
# 编辑 .env, 至少填写:
# OPENAI_API_KEY=你的key
# OPENAI_BASE_URL=https://api.openai.com/v1

# 启动服务
python -m api.main

# 测试接口 (新开终端)
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"user_id": "user_001", "message": "我想查询订单状态"}'
```

方式三：Java 版本

```
bash

cd java-impl

# 需要 Java 17+ 和 Maven 3.8+
mvn clean package -DskipTests

# 运行
java -jar target/smart-cs-agent-1.0.0.jar

# 或者直接用Maven运行
mvn spring-boot:run
```

方式四：Go 版本

```
bash

cd go-impl
```

```
# 需要 Go 1.22+
go mod tidy

# 运行
go run main.go

# 或者编译后运行
go build -o smart-cs-agent .
./smart-cs-agent
```

项目结构

```
code/
|
├─ python-impl/                               ← Python实现 (LangGraph + FastAPI)
|   ├── requirements.txt
|   ├── Dockerfile
|   └── agents/                                ← 核心Agent代码
|       ├── supervisor.py                     ← Supervisor编排Agent (核心)
|       ├── intent_router.py                  ← 意图识别Agent
|       ├── knowledge_rag.py                  ← RAG知识检索Agent
|       ├── ticket_handler.py                 ← 工单处理Agent
|       └── compliance_checker.py              ← 合规审查Agent
|   └── memory/                                ← 三层记忆系统
|       ├── working_memory.py                 ← 工作记忆 (进程内存)
|       ├── short_term.py                     ← 短期记忆 (Redis)
|       └── long_term.py                       ← 长期记忆 (FAISS向量库)
|   └── mcp/                                   ← MCP工具协议实现
|       ├── mcp_server.py                     ← JSON-RPC 2.0服务端
|       └── tools/                             ← 工具定义
|   └── tracing/                               ← OpenTelemetry追踪
|       └── otel_config.py                     ← 追踪配置 + Agent装饰器
```

└─ api/	← FastAPI接口层
└─ main.py	← REST API入口
─ java-impl/	← Java实现 (Spring AI + Spring Boot)
└─ pom.xml	
└─ Dockerfile	
└─ src/main/java/com/smartcs/	
└─ SmartCsApplication.java	← Spring Boot入口
└─ agent/	← Agent实现 (Supervisor + 4个子Agent)
└─ memory/	← 分层记忆 (ShortTerm + LongTerm)
└─ mcp/	← MCP工具服务端
└─ tracing/	← OpenTelemetry追踪
└─ config/	← Spring配置 + ChatController
─ go-impl/	← Go实现 (Gin + 自研LLM客户端)
└─ go.mod	
└─ main.go	← 程序入口
└─ Dockerfile	
└─ agent/	← Agent实现 (Supervisor + 4个子Agent)
└─ memory/	← 分层记忆 (Working + ShortTerm + LongTerm)
└─ llm/	← OpenAI兼容HTTP客户端
└─ mcp/	← MCP工具服务端
└─ tracing/	← 追踪
└─ api/	← Gin HTTP服务 + CORS中间件
└─ server.go	← 路由 + Handler

核心代码解析

Supervisor 编排核心逻辑 (Python)

这是整个系统最重要的部分，理解了这个代码，面试时就能讲清楚多Agent协作原理：

python

```

# python-impl/agents/supervisor.py

# 1. 定义全局状态（类似"黑板"，所有Agent共享）
class AgentState(TypedDict):
    messages: list[BaseMessage]    # 对话历史
    user_id: str                    # 用户ID
    intent: str                     # 识别到的意图
    sub_results: dict[str, Any]     # 各Agent处理结果
    compliance_passed: bool         # 合规是否通过
    final_response: str             # 最终回复

# 2. 构建有向图（定义Agent之间的流转关系）
def create_supervisor_graph():
    graph = StateGraph(AgentState)

    # 添加节点（每个Agent是一个节点）
    graph.add_node("supervisor_route", supervisor.route_decision)
    graph.add_node("knowledge_rag", knowledge_agent.process)
    graph.add_node("ticket_handler", ticket_agent.process)
    graph.add_node("compliance_check", compliance_agent.process)
    graph.add_node("synthesize", supervisor.synthesize_response)

    # 设置入口
    graph.set_entry_point("supervisor_route")

    # 条件路由（根据意图决定走哪条路）
    graph.add_conditional_edges(
        "supervisor_route",
        route_to_agent,          # 路由函数
        {
            "knowledge_rag": "knowledge_rag",
            "ticket_handler": "ticket_handler",
        }
    )

    # 所有Agent处理后都经过合规审查

```

```
graph.add_edge("knowledge_rag", "compliance_check")
graph.add_edge("ticket_handler", "compliance_check")
graph.add_edge("compliance_check", "synthesize")
graph.add_edge("synthesize", END)

return graph.compile(checkpointer=MemorySaver())
```

面试时怎么讲这段代码？

"我们用LangGraph的StateGraph构建了一个有向图，Supervisor作为中心节点负责路由，子Agent各司其职。所有回复都强制经过合规审查节点，这是金融场景的合规要求。MemorySaver提供检查点功能，支持对话断点续接。"

MCP工具协议实现

```
python

# python-impl/mcp/mcp_server.py

# MCP工具的核心：用装饰器注册工具，描述工具能力，让AI知道什么时候用
@server.register(
    name="order_query",
    description="查询订单信息，支持按订单号或用户ID查询",
    input_schema={
        "type": "object",
        "properties": {
            "order_id": {"type": "string", "description": "订单号"},
            "user_id": {"type": "string", "description": "用户ID"},
        },
    },
    category="order",
)
async def order_query(order_id: str = "", user_id: str = "") -> dict:
    return {
        "order_id": order_id or "ORD-20260401-001",
```

```
"status": "shipped",  
"amount": 299.00,  
"product": "智能理财产品A",  
}
```

面试准备材料

配套完整面试资料，帮你从"能看懂代码"到"面试时能流畅讲清楚"：

文档	内容说明
简历项目经历模板	STAR法则项目经历写法，覆盖Python/Java/Go不同岗位角度
STAR面试话术	"请介绍你的项目"等高频问题的标准回答模板
八股文题库	30+高频面试题 + 详细答案 + 追问应对策略
项目深度问答	面试官最爱问的20+深度问题 + 踩坑分享
架构设计文档	完整流程图、时序图、技术选型对比分析
代码讲解文档	核心模块逐行解析，设计模式说明
部署指南	Docker一键部署、生产环境配置、监控告警

常见面试问题预览

Q: 为什么用Supervisor模式而不是让Agent直接互相调用？

A: Supervisor模式的优势在于集中控制，便于追踪和调试；避免Agent之间形成循环依赖；Supervisor可以做全局优化，比如并行调度多个Agent；出错时有统一的错误处理和回退机制。

Q: 三层记忆的设计原则是什么？

A: 参考了人类认知的记忆模型。工作记忆对应当前注意力焦点，速度最快但容量有限；短期记忆用Redis实现30分钟TTL，保持对话上下文连贯性；长期记忆用向量数据库存储知识库和用户历史，支持语义相似度检索。

Q: MCP协议相比直接调用函数有什么优势？

A: MCP是标准化协议，工具描述用JSON Schema，AI可以自动发现和理解工具能力，不需要硬编码工具调用逻辑；支持动态工具注册，新增工具不需要修改Agent代码；协议层做了权限控制和参数验证。

参考项目

本项目设计参考了以下企业级开源项目，建议结合阅读：

项目	Stars	参考内容
AWS Agent Squad	7,500+	智能意图分类 + SupervisorAgent 设计
LangGraph Supervisor	—	Supervisor模式预构建库，官方最佳实践
Spring AI Alibaba	9,000+	Java多Agent编排，阿里巴巴生产实践
Eino (CloudWeGo)	10,300+	字节跳动Go企业级Agent框架
Multi-Agent Enterprise CRM	—	LangGraph + Kafka 生产级客服方案

架构设计文档

1. 系统总体架构

1.1 架构概览

本系统采用 **Supervisor编排模式**（中心化协调），由一个Supervisor节点统一调度4个专业子Agent（意图路由、知识检索、工单处理、合规审查）。所有回复必须经过合规审查节点，合规审查也可作为直接路由目标。

1.2 编排流程（Supervisor Pattern）

2. 核心组件设计

2.1 Agent设计原则

每个Agent遵循**单一职责原则**：

Agent	职责	输入	输出
Supervisor	编排调度、结果汇总	用户消息 + 全局State	路由决策 + 最终回复
意图路由	意图分类	用户消息	intent标签 + 置信度
知识检索	RAG问答	用户问题	基于文档的回答
工单处理	工单CRUD	用户需求	工单号 + 状态
合规审查	内容审查	Agent回复内容	通过/不通过 + 违规项

2.2 State设计

python

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages] # 对话消息 (reducer自动追加)
    user_id: str # 用户标识
    session_id: str # 会话标识
    intent: str # 识别到的意图
    sub_results: dict[str, Any] # 各Agent的处理结果
    compliance_passed: bool # 合规审查是否通过
    final_response: str # 最终回复
    current_agent: str # 当前执行的Agent
    retry_count: int # 重试次数
```

2.3 分层记忆架构

3. RAG检索流程

4. MCP工具协议

已注册工具：

工具名	分类	说明
order_query	order	查询订单状态、物流、金额
knowledge_search	knowledge	搜索企业知识库
ticket_create	ticket	创建客服工单
risk_check	compliance	风控接口，检查交易风险等级

工具注册示例（装饰器模式）：

python

```
@server.register(
    name="order_query",
    description="查询订单信息，支持按订单号或用户ID查询",
    input_schema={
        "type": "object",
        "properties": {
            "order_id": {"type": "string", "description": "订单号"},
            "user_id": {"type": "string", "description": "用户ID"},
        },
    },
    category="order",
)
async def order_query(order_id: str = "", user_id: str = "") -> dict:
    ...
```

5. 全链路追踪

Span层级结构

关键监控指标

指标	说明	告警阈值
P99延迟	99%请求的响应时间	> 5s
Agent错误率	各Agent的失败比例	> 5%
Token消耗/请求	平均每请求的Token数	> 5000
路由准确率	意图路由的正确率	< 85%
合规通过率	合规审查的通过比例	< 90%需关注

6. 技术选型对比

编排框架对比

维度	LangGraph (Python)	Spring AI (Java)	Gin + 原生并发 (Go)
编排模型	有向图StateGraph	手动顺序编排 + ChatClient	手动顺序编排
状态管理	TypedDict + Checkpoint	POJO	struct + sync.RWMutex
并行能力	asyncio (事件循环)	同步顺序调用	同步顺序调用
向量检索	FAISS (1536维)	64维哈希+余弦相似度	64维哈希+余弦相似度
LLM调用	LangChain ChatOpenAI	Spring AI ChatClient	自研OpenAI兼容HTTP客户端
合规审查	规则+LLM双阶段	规则+LLM双阶段 + PII脱敏	规则+LLM双阶段 + PII脱敏
生态	LangSmith/LangServe	Spring生态	标准库 + Gin + go-redis
适合团队	AI/数据团队	企业级Java团队	Go微服务团队
生产成熟度	高	中高	中

向量数据库对比

维度	FAISS	Milvus	Pinecone
部署方式	嵌入式	分布式	SaaS
数据规模	千万级	百亿级	百亿级
查询延迟	< 1ms	~10ms	~50ms
运维复杂度	低	中	零
成本	免费	中等	高
适合阶段	开发/小规模	生产	快速上线

代码讲解文档 — 核心模块逐行解析

本文档对三语言实现的核心模块进行详细讲解，帮助你在面试中清晰地描述代码设计。

1. Python实现核心讲解

1.1 Supervisor编排 (supervisor.py)

State定义 — 系统的"数据总线"

python

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    user_id: str
    session_id: str
    intent: str
    sub_results: dict[str, Any]
    compliance_passed: bool
    final_response: str
    current_agent: str
    retry_count: int
```

设计要点：

- `Annotated[list[BaseMessage], add_messages]`：LangGraph的reducer机制。当多个节点向messages写入时，`add_messages` 会自动追加而非覆盖
- `sub_results` 是一个dict：各Agent把结果写入不同key，避免互相覆盖
- `compliance_passed` 是布尔值：合规审查的结果，决定最终是否返回自动回复

Graph构建 — 编排的核心

python

```

graph = StateGraph(AgentState)

# 添加节点（每个节点是一个异步函数）
graph.add_node("supervisor_route", supervisor.route_decision)
graph.add_node("knowledge_rag", knowledge_agent.process)
graph.add_node("ticket_handler", ticket_agent.process)
graph.add_node("compliance_check", compliance_agent.process)
graph.add_node("synthesize", supervisor.synthesize_response)

# 设置入口
graph.set_entry_point("supervisor_route")

# 条件边：根据意图路由到不同Agent
graph.add_conditional_edges(
    "supervisor_route",
    route_to_agent, # 路由函数
    {
        "knowledge_rag": "knowledge_rag",
        "ticket_handler": "ticket_handler",
        "compliance_check": "compliance_check",
    },
)

# 固定边：所有业务Agent → 合规审查 → 汇总 → 结束
graph.add_edge("knowledge_rag", "compliance_check")
graph.add_edge("ticket_handler", "compliance_check")
graph.add_edge("compliance_check", "synthesize")
graph.add_edge("synthesize", END)

```

设计模式：

- **条件路由：** `add_conditional_edges` 根据运行时状态决定走哪条路径
- **汇聚点：** `compliance_check` 是汇聚节点，确保所有回复都经过审查
- **Checkpoint：** `MemorySaver()` 保存每步的State快照，支持断点恢复

1.2 RAG知识检索 (knowledge_rag.py)

完整RAG流程

```
python

async def process(self, state):
    original_query = messages[-1].content

    # Step 1: Query改写 (口语 → 检索友好)
    rewritten_query = await self.rewrite_query(original_query)

    # Step 2: 向量检索 (Top-5)
    raw_docs = await self.retrieve_documents(rewritten_query, top_k=5)

    # Step 3: 重排序 (Top-5 → Top-3)
    reranked_docs = await self.rerank_documents(rewritten_query, raw_docs,
top_k=3)

    # Step 4: 生成回答
    answer = await self.generate_answer(original_query, reranked_docs)
```

为什么先检索Top-5再排序到Top-3?

- 向量检索是近似检索，Top-5保证召回率
- 重排序用LLM做更精确的相关性判断，从5筛到3提升精确率
- 最终只注入3篇文档，控制Token消耗

1.3 合规审查 (compliance_checker.py)

两阶段审查机制

```
python

async def full_check(self, content: str) -> ComplianceResult:
    # Phase 1: 规则引擎 (毫秒级，零成本)
    rule_result = await self.rule_check(content)
```

```

# 高风险直接拦截，不走LLM
if not rule_result.passed and rule_result.risk_level in ("high",
"critical"):
    return rule_result

# Phase 2: LLM深度审查（秒级，有API成本）
llm_result = await self.llm_check(content)

# 合并两阶段结果
all_violations = rule_result.violations + llm_result.violations
final_passed = rule_result.passed and llm_result.passed

```

设计要点：

- 规则引擎是**高召回率的快筛**：宁可多报不能漏报
- LLM审查是**高精确率的精筛**：处理规则覆盖不了的场景
- 高风险**直接拦截不走LLM**：既省成本又降延迟

1.4 MCP工具协议 (mcp_server.py)

工具注册（装饰器模式）

python

```

@server.register(
    name="order_query",
    description="查询订单信息，支持按订单号或用户ID查询",
    input_schema={
        "type": "object",
        "properties": {
            "order_id": {"type": "string", "description": "订单号"},
            "user_id": {"type": "string", "description": "用户ID"},
        },
    },
    category="order",

```

```
)
async def order_query(order_id: str = "", user_id: str = "") -> dict:
    ...
```

JSON-RPC 2.0处理

python

```
async def handle_jsonrpc(self, request: dict) -> dict:
    method = request.get("method", "")
    params = request.get("params", {})

    if method == "tools/list":          # 工具发现
        result = self.list_tools(category=params.get("category"))
    elif method == "tools/call":        # 工具调用
        result = await self.call_tool(params["name"], params["arguments"])
    elif method == "ping":              # 心跳检测
        result = {"status": "ok"}
```

符合MCP规范的关键点：

- 使用 `tools/list` 和 `tools/call` 标准方法名
- 请求/响应遵循JSON-RPC 2.0格式 (`jsonrpc/method/params/id/result/error`)
- 工具声明包含 `inputSchema` 用于参数校验

1.5 OpenTelemetry追踪 (otel_config.py)

追踪装饰器

python

```
def trace_agent_call(agent_name: str) -> Callable:
    def decorator(func):
        async def wrapper(*args, **kwargs):
            with tracer.start_as_current_span(span_name) as span:
                span.set_attribute("agent.name", agent_name)
```

```
start_time = time.time()

result = await func(*args, **kwargs)

span.set_attribute("agent.duration_ms", duration_ms)
span.set_attribute("agent.success", True)
return result
```

设计模式：装饰器模式 + AOP（面向切面编程）。业务代码完全不需要关心追踪逻辑，只需要加一行装饰器。

2. Java实现核心讲解

2.1 Supervisor编排 (SupervisorAgent.java)

```
java

public AgentState orchestrate(AgentState state) {
    return tracer.trace("supervisor", "orchestrate", () -> {
        // Step 1: 意图路由
        AgentState routedState = intentRouter.process(state);

        // Step 2: 分发到子Agent
        AgentState processedState = dispatchToAgent(routedState, intent);

        // Step 3: 合规审查
        AgentState checkedState = complianceAgent.process(processedState);

        // Step 4: 汇总
        return synthesize(checkedState);
    });
}
```

Java特有设计：

- 使用 `Supplier<T>` 函数式接口传入追踪逻辑，实现类似Python装饰器的效果
- `switch` 表达式（Java 21）实现路由分发，比if-else更清晰
- `ConcurrentHashMap` 保证工单存储的线程安全

2.2 依赖注入（Spring IoC）

```
java

@Component
public class SupervisorAgent {
    private final IntentRouterAgent intentRouter;
    private final KnowledgeRAGAgent knowledgeAgent;
    // ... 构造器注入
}
```

所有Agent通过Spring容器管理，Supervisor通过构造器注入持有所有子Agent的引用。面试中可以讨论：为什么用构造器注入而不是@Autowired？（答：不可变性 + 必须依赖 + 更好的测试性）

2.3 MCP工具服务端 — 真实工具执行 (MCPToolServer.java)

```
java

// 每个工具注册一个真实处理函数
private final Map<String, Function<Map<String, Object>, Object>> handlers = new
HashMap<>();

private Object handleOrderQuery(Map<String, Object> args) {
    String orderId = (String) args.getOrDefault("order_id", "");
    Map<String, Object> result = new HashMap<>();
    result.put("order_id", orderId.isEmpty() ? "ORD-20260401-001" : orderId);
    result.put("status", "shipped");
    result.put("amount", 299.00);
}
```

```
    return result;
}
```

设计要点:

- `handlers` Map将工具名映射到 `Function<Map, Object>` 处理函数
- 4个工具均有真实业务逻辑: `order_query`、`knowledge_search`、`ticket_create`、`risk_check`
- `risk_check` 根据金额阈值分级: `>50k=high`, `>10k=medium`
- 调用日志包含 `success`、`duration_ms`、`error` 字段, 便于监控

2.4 KnowledgeRAGAgent — RAG管线 (KnowledgeRAGAgent.java)

```
java
```

```
public AgentState process(AgentState state) {
    String originalQuery = state.getUserMessage();

    // Step 1: Query改写 (口语 → 检索友好)
    String rewrittenQuery = rewriteQuery(originalQuery);

    // Step 2: 向量检索 (Top-5)
    List<Map<String, Object>> rawDocs = longTermMemory.search(rewrittenQuery,
5);

    // Step 3: 重排序 (Top-5 → Top-3)
    List<Map<String, Object>> rerankedDocs = rerankDocuments(rewrittenQuery,
rawDocs, 3);

    // Step 4: 构建上下文 + 生成回答
    String answer = chatClient.prompt()
        .system(RAG_SYSTEM_PROMPT)
        .user(String.format("用户问题: %s\n\n检索到的参考文档:\n%s",
originalQuery, context))
        .call()
```

```
        .content();  
    }  
}
```

为什么先检索Top-5再排序到Top-3?

- 向量检索是近似检索，Top-5保证召回率
- 重排序用LLM做更精确的相关性判断，从5筛到3提升精确率
- 最终只注入3篇文档，控制Token消耗

Query改写：将口语化问题（如“怎么退钱啊”）改写为检索友好查询（如“退款政策 申请流程”），提升检索命中率。改写失败时安全降级为原始查询。

重排序：LLM对候选文档的相关性评分，返回最相关的索引号。解析失败时回退到Top-K子集。

2.5 ComplianceCheckerAgent — 两阶段审查 (ComplianceCheckerAgent.java)

```
java  
  
public ComplianceResult fullCheck(String content) {  
    // Phase 1: 规则引擎快速检查（毫秒级，不调用LLM）  
    ComplianceResult ruleResult = ruleCheck(content);  
  
    // 高风险直接拦截，不走LLM（省成本+降延迟）  
    if (!ruleResult.passed() && isHighRisk(ruleResult.riskLevel())) {  
        return ruleResult;  
    }  
  
    // Phase 2: LLM深度审查（处理规则无法覆盖的场景）  
    ComplianceResult llmResult = llmCheck(content);  
  
    // 合并两阶段结果  
    return new ComplianceResult(  
        ruleResult.passed() && llmResult.passed(),  
        maxRisk(ruleResult.riskLevel(), llmResult.riskLevel()),  
        allViolations, llmResult.suggestions(),  
    );  
}
```

```
ruleResult.sanitizedContent()
    );
}
```

ComplianceResult结构化返回：

```
java

public record ComplianceResult(
    boolean passed,           // 是否通过
    String riskLevel,         // low | medium | high | critical
    List<String> violations,   // 违规项列表
    List<String> suggestions, // 修改建议
    String sanitizedContent   // PII脱敏后的内容
) {}
```

风险分级逻辑：

- PII泄露 + 违规金融用语 → `critical`
- 任一项 → `high`
- 其他 → `medium`

PII脱敏：手机号、身份证号、银行卡号自动掩码（前3位 + `*` + 后3位）。

2.6 LongTermMemoryService — 分块 + 向量检索 + 持久化 (LongTermMemoryService.java)

```
java

// 文档分块：按句子分割，支持重叠窗口
public List<String> chunkText(String text, int chunkSize, int overlap) {
    String[] sentences = text.split("(?<=[。！？.!?\\n])\\s*");
    // 当累积长度超过chunkSize时切分，保留overlap字符作为上下文
}

// 混合检索：关键词得分 + 向量相似度（权重3x）
public List<Map<String, Object>> search(String query, int topK) {
```



```

double[] queryEmbedding = simpleEmbed(query);
return documents.stream()
    .map(doc -> {
        long keywordScore =
queryTerms.stream().filter(content::contains).count();
        double vectorScore = cosineSimilarity(queryEmbedding, docEmb);
        doc.put("score", keywordScore + vectorScore * 3);
    })
    .sorted(by score desc)
    .limit(topK)
    .collect(toList());
}

```

向量实现： `simpleEmbed()` 生成64维哈希向量（占位实现，生产应替换为真实Embedding模型）。`cosineSimilarity()` 计算余弦相似度。

持久化： `saveToDisk()` 将文档列表序列化为JSON写入 `knowledge_base.json`（不含向量），启动时 `loadFromDisk()` 自动加载并重新计算向量。

3. Go实现核心讲解

3.1 Supervisor编排设计

```

go

func (s *SupervisorAgent) Orchestrate(state *State) *State {
    return tracing.TraceFunc("supervisor", "orchestrate", func() *State {
        // Step 1: 意图路由
        routedState := s.intentRouter.Process(state)

        // Step 2: 分发到子Agent（同步顺序执行）
        processedState := s.dispatch(routedState)

        // Step 3: 合规审查
    })
}

```

```

        checkedState := s.complianceAgent.Process(processedState)

        // Step 4: 汇总
        return s.synthesize(checkedException)
    })
}

```

Go的并发基础设施（当前为顺序执行，可按需启用并行）：

- goroutine创建成本极低（~2KB栈空间），可为每个Agent请求创建goroutine
- `sync.RWMutex` 实现读写锁，工作记忆支持并发读、互斥写
- channel可用于Agent间异步消息传递

3.2 泛型追踪函数

```

go

func TraceFunc[T any](agentName, method string, fn func() T) T {
    start := time.Now()
    result := fn()
    elapsed := time.Since(start)
    RecordMetric(agentName, elapsed.Milliseconds(), true)
    return result
}

```

Go 1.18+的泛型让追踪函数可以适用于任何返回类型，类似Python的装饰器效果。

3.3 自研LLM客户端 (llm/client.go)

Go版本没有依赖LangChain等框架，而是自研了一个轻量级OpenAI兼容HTTP客户端：

```

go

type Client struct {
    apiKey string
    baseURL string
}

```

```

    model    string
    http     *http.Client
}

func (c *Client) ChatCompletion(messages []Message) (string, error) {
    reqBody := ChatRequest{Model: c.model, Messages: messages}
    body, _ := json.Marshal(reqBody)

    req, _ := http.NewRequest("POST", c.baseURL+"/chat/completions",
bytes.NewReader(body))
    req.Header.Set("Authorization", "Bearer "+c.apiKey)

    resp, _ := c.http.Do(req)
    // ... 解析ChatResponse
}

func (c *Client) ChatCompletionWithSystem(systemPrompt, userMessage string)
(string, error) {
    return c.ChatCompletion([]Message{
        {Role: "system", Content: systemPrompt},
        {Role: "user", Content: userMessage},
    })
}

```

为什么自研而不是用框架？

- Go生态缺少LangChain等成熟AI框架
- OpenAI API本身足够简单，直接HTTP调用即可
- 环境变量配置： `OPENAI_API_KEY`、 `OPENAI_BASE_URL`、 `MODEL_NAME`
- 30秒超时，支持任意OpenAI兼容端点（如Azure、本地模型）

3.4 KnowledgeRAGAgent — 完整RAG管线 (agent/knowledge_rag.go)

```
go
```

```

func (a *KnowledgeRAGAgent) Process(state *State) *State {
    originalQuery := state.UserMessage

    // Step 1: Query改写 (口语 → 检索友好)
    rewrittenQuery := a.rewriteQuery(originalQuery)

    // Step 2: 向量检索 (Top-5)
    rawDocs := a.longTermMemory.Search(rewrittenQuery, 5)

    // Step 3: 重排序 (Top-5 → Top-3)
    docs := a.rerankDocuments(rewrittenQuery, rawDocs, 3)

    // Step 4: LLM生成回答
    answer := a.generateAnswer(originalQuery, context)
}

```

Query改写与重排序与Java/Python实现对齐：

- `rewriteQuery()`：LLM将口语化查询改写为检索友好形式
- `rerankDocuments()`：LLM对Top-5候选重排序取Top-3，返回逗号分隔的索引号
- 两个方法均有降级策略：LLM不可用时返回原始查询 / Top-K子集

3.5 ComplianceCheckerAgent — PII脱敏 + LLM审查 (agent/compliance_checker.go)

```

go

// Phase 1: 规则引擎 - PII检测 + 敏感词
func (a *ComplianceCheckerAgent) ruleCheck(content string) complianceResult {
    // 检测违规金融用语：保证收益、稳赚不赔、零风险...
    // 检测PII信息：手机号(1[3-9]\d{9})、身份证号(\d{17}[\dXx])、银行卡号(\d{16,19})
    // 脱敏：前3位 + *** + 后3位
}

// Phase 2: LLM深度审查
func (a *ComplianceCheckerAgent) llmCheck(content string) complianceResult {

```

```

    response, _ := a.llmClient.ChatCompletionWithSystem(complianceSystemPrompt,
prompt)
    // 解析JSON返回: {passed, risk_level, violations, suggestions}
}

```

PII脱敏规则（与Python/Java一致）：

- 手机号 13812345678 → 138***5678
- 身份证号 110101199001011234 → 110*****1234
- 银行卡号 6222021234567890123 → 622*****0123

高风险拦截策略：PII + 违规用语同时出现 → `critical`，直接拦截不走LLM。

3.6 LongTermMemory — 向量检索 + 分块 + 持久化 (memory/long_term.go)

```

go

// 混合检索：关键词 + 向量相似度（权重3x）
func (m *LongTermMemory) Search(query string, topK int) []Document {
    queryEmbedding := simpleEmbed(query)
    for _, doc := range m.documents {
        keywordScore := countKeywordMatches(queryTerms, doc.Content)
        vectorScore := cosineSimilarity(queryEmbedding, doc.Embedding)
        totalScore := float64(keywordScore) + vectorScore*3
    }
}

// 文档分块：按句子分割，支持重叠窗口
func ChunkText(text string, chunkSize, overlap int) []string {
    sentences := splitSentences(text) // 按。！？\n分割
    // 累积长度超过chunkSize时切分，保留overlap字符
}

// 磁盘持久化：JSON序列化（不含向量）

```

```
func (m *LongTermMemory) SaveToDisk() { ... }
func (m *LongTermMemory) loadFromDisk() { ... }
```

默认知识库： `NewLongTermMemory()` 自动加载3条默认文档（理财FAQ、退款政策、开户流程），并从 `knowledge_base.json` 恢复持久化数据。

向量实现： `simpleEmbed()` 生成64维哈希向量，`cosineSimilarity()` 计算余弦相似度。与Java实现完全对齐。生产环境应替换为真实的Embedding模型（如text-embedding-3-small）。

3.7 ShortTermMemory — Redis集成 + 上下文窗口 (memory/short_term.go)

```
go

// Redis连接失败自动降级为内存存储
func NewShortTermMemory(redisURL string) *ShortTermMemory {
    client := redis.NewClient(opt)
    if err := client.Ping(ctx).Err(); err != nil {
        log.Printf("Redis connection failed, using in-memory fallback")
    }
}

// 按Token估算裁剪对话历史
func (m *ShortTermMemory) GetContextWindow(sessionID string, maxTokens int)
string {
    history := m.GetHistory(sessionID)
    for i := len(history) - 1; i >= 0; i-- {
        msgTokens := len(msgText) / 2 // 粗略估算：2字符≈1token
        if estimatedTokens+msgTokens > maxTokens { break }
    }
}
```

降级策略： Redis连接失败 → 自动切换内存存储，日志提示但不阻塞业务。

`AddMessage` / `GetHistory` 均实现双路径。

3.8 CORS中间件 (api/server.go)

```
go

func corsMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Header("Access-Control-Allow-Origin", "*")
        c.Header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS")
        c.Header("Access-Control-Allow-Headers", "Content-Type, Authorization,
X-Requested-With")
        c.Header("Access-Control-Max-Age", "86400")

        if c.Request.Method == "OPTIONS" {
            c.AbortWithStatus(http.StatusNoContent)
            return
        }
        c.Next()
    }
}
```

OPTIONS预检请求直接返回204，避免前端跨域调用被拦截。

4. 设计模式总结

模式	应用位置	说明
回退模式	意图路由Agent	LLM分类为主，关键词匹配为降级回退
责任链模式	合规审查	规则引擎 → LLM审查，逐层过滤，高风险直接拦截
装饰器/AOP	追踪 (Python装饰器/Go TraceFunc)	无侵入地添加追踪能力
注册模式	MCP工具注册	装饰器(Python)/直接注册(Java/Go)声明工具能力
状态模式	工单状态流转	created → processing → resolved

面试时提到设计模式可以加分，但要结合具体场景说明为什么用这个模式，而不是为了用而用。

部署指南

1. 本地开发环境

Python版本

```
bash

cd python-impl
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
# 编辑 .env 填入你的API Key
python -m api.main
```

访问: <http://localhost:8000/docs> (Swagger UI)

Java版本

```
bash

cd java-impl
mvn clean package -DskipTests
java -jar target/smart-cs-agent-1.0.0.jar
```

访问: <http://localhost:8080/api/health>

Go版本

```
bash

cd go-impl
go mod tidy
go run main.go
```

访问: <http://localhost:8090/health>

2. Docker部署

单服务启动

```
bash

# Python
cd python-impl
docker build -t smart-cs-python .
docker run -p 8000:8000 --env-file .env smart-cs-python

# Java
cd java-impl
mvn clean package -DskipTests
docker build -t smart-cs-java .
docker run -p 8080:8080 -e OPENAI_API_KEY=xxx smart-cs-java

# Go
cd go-impl
docker build -t smart-cs-go .
docker run -p 8090:8090 smart-cs-go
```

Docker Compose 一键启动

```
bash

docker-compose up -d
```

3. API接口说明

所有三个版本提供统一的REST API:

POST /api/chat — 聊天接口

```
json

// Request
{
  "message": "我想了解一下理财产品A",
  "user_id": "user_001",
  "session_id": "optional-session-id"
}

// Response
{
  "response": "关于理财产品A...",
  "session_id": "xxx",
  "intent": "knowledge_rag",
  "compliance_passed": true
}
```

GET /api/history/{session_id} — 对话历史

GET /api/tools — MCP工具列表

POST /api/tools/call — MCP工具调用

Python/Java接受直接JSON格式，Go接受JSON-RPC 2.0格式：

```
json

// Request (Python / Java)
{
  "name": "order_query",
  "arguments": {"order_id": "ORD-20260401-001"}
}

// Request (Go - JSON-RPC 2.0)
{
```

```
"jsonrpc": "2.0",
"method": "tools/call",
"params": {"name": "order_query", "arguments": {"order_id": "ORD-20260401-001"}},
"id": 1
}

// Response
{
  "success": true,
  "result": {"order_id": "ORD-20260401-001", "status": "shipped", "amount": 299.00},
  "duration_ms": 12.5
}
```

GET /api/metrics — 系统指标

GET /health — 健康检查

4. 环境变量说明

变量	说明	默认值	适用语言
OPENAI_API_KEY	LLM API密钥	无	全部
OPENAI_BASE_URL	API端点	https://api.openai.com/v1	Python/Go
MODEL_NAME	模型名称	gpt-4o	Python/Go
REDIS_URL	Redis地址	redis://localhost:6379/0	Python/Go
REDIS_HOST	Redis主机	localhost	Java
REDIS_PORT	Redis端口	6379	Java
OTEL_SERVICE_NAME	追踪服务名	smart-cs-multi-agent	Python
OTEL_EXPORTER_OTLP_ENDPOINT	OTLP端点	http://localhost:4317	Python

Go版本注意事项：

- LLM客户端通过环境变量 `OPENAI_API_KEY`、`OPENAI_BASE_URL`、`MODEL_NAME` 配置，支持任意OpenAI兼容端点
- Redis连接失败时自动降级为内存存储，不影响服务启动
- API默认启用CORS中间件（`Access-Control-Allow-Origin: *`），前端可直接跨域调用
- 长期记忆自动加载默认知识库并从 `knowledge_base.json` 恢复持久化数据

八股文题库 — 多Agent系统 & RAG & 大模型应用

30+高频面试题，每题配详细答案和追问应对。按主题分类，从基础到进阶。

一、多Agent架构基础 (8题)

Q1: 单Agent vs 多Agent，什么时候用多Agent？

答：

单Agent适合任务单一、工具集小、不需要专业化分工的场景（如简单QA机器人）。多Agent适合以下场景：

1. **任务复杂度高**：需要拆解为多个子任务，各自需要不同的Prompt和工具集
2. **专业化分工**：不同子任务需要不同的"专家"——如金融合规和产品咨询需要完全不同的知识和判断标准
3. **上下文隔离**：单Agent把所有Prompt和工具塞进一个上下文窗口，Token消耗大且容易混淆
4. **并行处理**：多个子任务可以并发执行，降低总延迟

追问：多Agent的缺点是什么？

- 系统复杂度增加：编排、通信、状态同步都需要额外设计
- 延迟增加：Agent间的路由和消息传递有额外开销
- 调试困难：一个请求可能经过3-4个Agent，问题定位需要全链路追踪
- 成本可能更高：多次LLM调用（意图分类一次、业务处理一次、合规审查一次）

Q2: Supervisor模式 vs Peer-to-Peer模式的优劣？

答：

维度	Supervisor模式	Peer-to-Peer模式
控制方式	中央协调者分发和汇总	Agent间直接通信
适合场景	需要统一审查的场景（客服、金融）	需要协商决策的场景（研究、辩论）
优点	流程可控、容易加审查环节	灵活、去中心化、无单点故障
缺点	Supervisor是单点、所有请求都经过它	难以保证一致性、调试复杂
状态管理	集中式State，由Supervisor维护	分布式State，各Agent各自维护

在智能客服场景选择Supervisor的原因：**所有回复必须经过合规审查**，这天然需要一个中央节点来保证。

追问：Supervisor成为性能瓶颈怎么办？

- 1. Supervisor只做路由决策（轻量级），不做业务处理
- 2. 对Supervisor的LLM调用做缓存——相似问题的路由决策可以cache
- 3. 多实例部署Supervisor，按session_id做一致性哈希分流

Q3: 解释一下ReAct框架的原理？

答：

ReAct = Reasoning + Acting。核心思想是让LLM在推理和行动之间交替执行：

循环：

- 1. Thought：LLM思考当前状况，决定下一步
- 2. Action：调用一个工具（搜索、查询、计算等）
- 3. Observation：获取工具返回的结果

4. 回到步骤1，基于新Observation继续推理

直到：LLM判断可以给出最终答案

与纯Chain-of-Thought的区别：CoT只有推理没有行动，ReAct可以在推理过程中调用外部工具获取实时信息。

追问：ReAct有什么问题？怎么改进？

- **死循环**：Agent可能在Thought和Action之间无限循环。解决：设最大循环次数 + 超时熔断
- **工具选择错误**：LLM可能选错工具。解决：优化工具描述、Few-shot示例
- **效率低**：每次循环都要调用LLM。解决：Plan-then-Execute模式，先一次性规划所有步骤

Q4: 多Agent间怎么通信？有哪些通信模式？

答：

1. **共享状态 (Shared State)**：所有Agent读写同一个State对象。LangGraph的主要模式。优点是简单直接，缺点是需要处理并发写入冲突。
2. **消息传递 (Message Passing)**：Agent之间通过消息队列通信。适合分布式部署。如Kafka、RabbitMQ。
3. **工具调用 (Agent-as-Tool)**：一个Agent把另一个Agent当作工具来调用。AWS Agent Squad的SupervisorAgent就是这种模式。
4. **黑板模式 (Blackboard)**：所有Agent向一个共享"黑板"写入信息，其他Agent可以读取。适合需要多Agent贡献知识的场景。

我们的项目采用**共享状态模式**，原因是：

- 所有Agent在同一个进程内运行，无需网络通信
- LangGraph的StateGraph天然支持这种模式
- Supervisor可以直接读取所有子Agent的结果进行汇总

Q5: 什么是Human-in-the-Loop？在Agent系统中怎么实现？

答：

Human-in-the-Loop (HITL) 指在Agent自动化流程中插入人工审核/决策节点。

实现方式：

1. **LangGraph Breakpoint**：在Graph的特定节点设置interrupt，执行到该节点时暂停，等待人工输入后继续
2. **Checkpoint + 恢复**：Agent执行到断点时，将完整State持久化（checkpoint），人工审核后恢复执行
3. **审批队列**：敏感操作（如大额退款）发送到审批队列，人工审批后回调Agent继续

在我们的项目中，当合规审查Agent检测到高风险内容时，会自动创建工单并暂停自动回复，等待人工审核。

Q6: Agent的幻觉问题怎么解决？

答：

Agent幻觉指LLM生成与事实不符的内容。在客服场景下特别危险——比如承诺不存在的退款政策。

多层防御策略：

1. **RAG约束**：在Prompt中明确要求"只基于检索到的文档回答，不要编造"，并提供Few-shot反例
 2. **合规审查Agent**：专门检测越权承诺和虚假信息
 3. **引用追踪**：回答中标注引用来源，方便人工验证
 4. **置信度阈值**：当RAG检索相似度低于阈值时，不自动回答，转人工
 5. **自我纠正 (Self-Correction)**：让Agent对自己的输出进行二次审视，检查是否与文档矛盾
-

Q7: Token消耗优化有哪些方案？

答：

1. **分层记忆**：不是每次都把全部历史塞进prompt。工作记忆只存关键状态，短期记忆做滑动窗口，长期记忆用向量检索按需取用
2. **Prompt压缩**：用LLM对历史对话做摘要，代替完整历史
3. **缓存**：相似问题的路由决策和检索结果做缓存
4. **模型分级**：意图路由用小模型（便宜快速），知识生成用大模型（质量优先）
5. **早退机制**：简单问题（FAQ命中）直接返回，不走完整Agent链路

在我们的项目中，分层记忆使Token消耗降低了40%。

Q8: 多Agent系统的容错和降级策略？

答：

1. **超时熔断**：每个Agent设超时上限（如10秒），超时自动降级
2. **重试策略**：对可重试错误（如API限流）做指数退避重试，最多3次
3. **降级回复**：Agent异常时返回兜底话术 + 自动创建工单转人工
4. **隔舱设计**：一个子Agent故障不影响其他Agent执行
5. **Checkpoint恢复**：对长流程做检查点，异常后可从检查点恢复

二、RAG知识检索 (8题)

Q9: 描述一下完整的RAG流程？

答：

用户问题 → Query改写 → 向量化 → 向量检索(Top-K) → 重排序(Re-rank) → 上下文注入 → LLM生成 → 引用标注

1. **Query改写**：将口语化问题转为检索友好的查询（如"怎么退钱"→"退款政策 申请流程 时限"）
2. **向量化**：用Embedding模型将Query转为向量

- 3. 向量检索：在向量数据库中做近似最近邻搜索，取Top-K候选
- 4. 重排序：用Cross-Encoder或LLM对候选文档按相关性重排
- 5. 上下文注入：将排序后的Top文档拼接 Prompt 中
- 6. LLM生成：基于上下文生成回答
- 7. 引用标注：标注回答引用了哪些文档

Q10: 文档分块（Chunking）有哪些策略？各自优缺点？

答：

策略	原理	优点	缺点
固定长度	按字符/Token数切分	简单快速	可能切断句子语义
语义分块	按段落/句子分割	保持语义完整	块大小不均匀
滑动窗口	固定长度+重叠	避免边界信息丢失	有冗余存储
递归分割	先按段落，段落太长则按句子	兼顾语义和大小	实现复杂

我们的项目采用**固定长度(512 tokens) + 重叠窗口(128 tokens)**，同时优先按段落分割。

追问：重叠窗口的overlap设多大合适？

一般设chunk_size的10%-25%。太小会丢失边界信息，太大会增加存储和检索成本。我们用128/512=25%。

Q11: 向量数据库选型（FAISS vs Milvus vs Pinecone）？

答：

维度	FAISS	Milvus	Pinecone
部署	单机库，需自管理	分布式，云原生	全托管SaaS
性能	最快（毫秒级）	快（10ms级）	中等（API延迟）
规模	千万级	百亿级	百亿级
成本	免费	中等	最高
运维	需自行管理	需K8s运维	零运维

选型建议：

- 开发/测试：FAISS（零依赖，快速验证）
- 小规模生产（<1000万向量）：FAISS也够用
- 大规模生产（>1亿向量）：Milvus
- 不想运维：Pinecone（但注意数据合规）

Q12: 检索效果不好怎么优化？

答：

1. **Query改写**：用LLM改写用户问题，补充专业术语
2. **混合检索**：向量检索 + BM25关键词检索，合并结果
3. **重排序（Re-rank）**：用Cross-Encoder对Top-K结果重排
4. **优化Embedding模型**：用领域微调的Embedding模型
5. **优化分块策略**：调整chunk_size和overlap
6. **多路召回**：同时用不同的检索策略，取并集
7. **反馈学习**：根据用户反馈标注好/坏case，持续优化

Q13: RAG vs Fine-tuning，怎么选？

答：

维度	RAG	Fine-tuning
知识更新	实时（更新文档即可）	需要重新训练
成本	检索+生成开销	训练+部署开销
可解释性	高（可引用来源）	低（黑箱）
幻觉控制	更好（有文档约束）	较差
适合场景	知识频繁更新、需要引用	风格/格式固定、领域深度理解

客服场景选RAG：产品信息频繁更新，需要引用政策文档，合规要求可追溯。

Q14: Embedding模型怎么选？

答：

主流选择：

- OpenAI text-embedding-3-large：通用效果好，1536维
- BGE系列：中文效果最好的开源模型
- Cohere Embed v3：多语言支持好
- Sentence-Transformers：本地部署首选

选型考虑：语言（中文需要中文优化模型）、维度（越高越精确但越慢）、是否支持本地部署（数据安全要求）。

Q15: 什么是Hybrid Search（混合检索）？

答：

结合稠密向量检索（语义相似度）和稀疏检索（BM25关键词匹配）的检索方法。

原理：

- 稠密检索擅长理解语义（"怎么退钱" $\approx$ "退款流程"）
- 稀疏检索擅长精确匹配（专有名词、订单号）
- 两者结合：先各自检索Top-K，再用RRF（Reciprocal Rank Fusion）合并排序

实际效果通常比纯向量检索提升5-15%。

Q16: 上下文窗口不够用怎么办？

答：

1. **压缩上下文**：用LLM对检索结果做摘要，只保留关键信息
 2. **分层注入**：最相关的文档完整注入，次相关的只注入摘要
 3. **Map-Reduce**：对每个文档分别生成子回答，最后合并
 4. **长上下文模型**：使用128K+窗口的模型（如GPT-4o、Claude）
 5. **筛选策略**：严格控制注入文档数量，只注入相关性最高的2-3篇
-

三、MCP工具协议 (4题)

Q17: MCP协议是什么？和传统API调用有什么区别？

答：

MCP（Model Context Protocol）是Anthropic开源的标准协议，定义了AI Agent与外部工具/数据源的通信方式。

与传统API调用的区别：

维度	传统API	MCP
集成方式	N个Agent × M个工具 = N×M个适配器	N+M个标准化连接
工具发现	硬编码	动态发现 (tools/list)
协议	各自定义	统一JSON-RPC 2.0
扩展性	加一个工具改N个Agent	加一个工具所有Agent自动可用

类比：MCP之于AI工具，就像USB-C之于硬件设备——统一接口标准。

Q18: MCP协议包含哪些核心方法？

答：

- 1. `tools/list`：工具发现，Agent查询可用工具列表和参数Schema
- 2. `tools/call`：工具调用，Agent传入工具名和参数，获取结果
- 3. `resources/list`：资源发现，列出可用的数据资源
- 4. `resources/read`：资源读取
- 5. `prompts/list` + `prompts/get`：提示模板管理

传输方式：JSON-RPC 2.0 over stdio / HTTP+SSE

Q19: 在项目中怎么实现MCP？

答：

我们实现了一个MCPToolServer类：

- 1. **工具注册**：通过装饰器/注解注册工具函数，声明name、description、inputSchema
- 2. **工具发现**：暴露 `/api/tools` 端点，Agent启动时自动获取可用工具列表
- 3. **工具调用**：Agent通过 `tools/call` 方法传入参数，MCPToolServer路由到对应handler执行
- 4. **日志记录**：每次调用记录工具名、参数、耗时、成功/失败，用于追踪分析

Q20: 工具调用失败怎么处理？

答：

- 1. **重试**：可重试错误（超时、限流）做指数退避重试
- 2. **降级**：工具不可用时返回降级结果（如缓存的旧数据）
- 3. **错误传播**：将错误信息传回Agent，让Agent决定下一步（换工具或告知用户）
- 4. **熔断器**：连续失败超过阈值，自动熔断该工具，避免雪崩

四、分层记忆系统 (4题)

Q21: 分层记忆系统的三层分别是什么？

答：

层级	存储介质	生命周期	延迟	用途
工作记忆	进程内存	单次请求	<1ms	Agent当前推理状态
短期记忆	Redis	30分钟TTL	1-5ms	最近20轮对话上下文
长期记忆	向量数据库	永久	10-50ms	知识库、用户画像、历史工单

三层协作：

- 1. Supervisor先查工作记忆获取当前状态
- 2. 需要对话上下文时查短期记忆
- 3. 需要知识或用户画像时查长期记忆
- 4. 重要中间结果从工作记忆提升到短期记忆（跨轮保留）

Q22: 短期记忆的淘汰策略？

答：

1. **滑动窗口**：只保留最近N轮（我们用20轮）
2. **TTL过期**：Redis自动过期（30分钟无活动）
3. **Token预算**：总Token数超过预算时，从最早的消息开始淘汰
4. **重要性权重**：用户首次描述的问题比中间确认消息重要，可优先保留

追问：缓存击穿/雪崩怎么处理？

- 击穿：热点session的缓存过期时大量请求打到后端。解决：互斥锁 + 永不过期(后台异步更新)
- 雪崩：大量session同时过期。解决：TTL加随机偏移量（如30min±5min）

Q23: 长期记忆什么时候写入？

答：

1. **知识库批量导入**：系统启动时或管理员上传文档时
2. **对话结束摘要**：会话结束后，将对话摘要和关键实体写入长期记忆
3. **工单信息**：工单创建/更新时写入
4. **用户画像更新**：从对话中提取的用户偏好、常见问题模式

写入策略：异步写入，不阻塞主流程。使用消息队列缓冲写入请求。

Q24: 记忆系统在分布式环境下怎么保证一致性？

答：

- **工作记忆**：进程内，无需分布式一致性，但需要按session_id做请求亲和（sticky session）
- **短期记忆**：Redis单节点或Redis Cluster，依赖Redis自身的一致性保证
- **长期记忆**：向量数据库（Milvus）支持分布式一致性，写入后需要等flush完成才对读可见

关键设计：session_id的一致性路由——同一个session的所有请求路由到同一个实例，避免工作记忆不一致。

五、全链路追踪 (4题)

Q25: 为什么Agent系统特别需要全链路追踪?

答:

传统微服务的追踪主要看HTTP调用链。Agent系统更复杂:

1. **路由决策不确定**: 同一个问题可能路由到不同Agent
2. **LLM输出不确定**: 相同输入可能产生不同输出
3. **多Agent串行/并行**: 一个请求可能触发3-4个Agent调用
4. **Token消耗不透明**: 不追踪就无法知道钱花在哪里

追踪Agent系统需要额外记录: 每个Agent的路由决策原因、LLM的输入/输出、Token消耗、工具调用参数和结果。

Q26: OpenTelemetry在Agent系统中怎么埋点?

答:

```
python

@trace_agent_call("knowledge_rag")
async def process(self, state):
    # 自动创建Span, 记录:
    # - agent.name
    # - agent.duration_ms
    # - agent.success
    ...
```

Span层级:

```
Root Span: user_request
├── supervisor.route_decision
├── knowledge_rag.process
```

```
|   |— rag.query_rewrite
|   |— rag.vector_search
|   └─ rag.generate_answer
|— compliance_checker.process
|   |— compliance.rule_check
|   └─ compliance.llm_check
└─ supervisor.synthesize
```

Q27: 追踪数据怎么用于优化系统？

答：

1. 延迟分析：哪个Agent最慢？是LLM调用慢还是向量检索慢？
2. 路由准确率：统计意图路由的准确率，发现路由错误的Pattern
3. Token消耗分布：哪个Agent消耗Token最多？是否有优化空间？
4. 错误率监控：哪个Agent错误率最高？错误原因是什么？
5. Badcase分析：用户投诉时可以回溯完整链路

Q28: LangSmith和OpenTelemetry的关系？

答：

- OpenTelemetry：通用的分布式追踪标准，适用于任何系统
- LangSmith：LangChain团队的LLM专用观测平台，专门为LLM应用优化

LangSmith现在支持OpenTelemetry格式，可以用OTEL标准收集Trace再导入LangSmith做LLM专用分析。

推荐：用OpenTelemetry做基础设施级追踪（延迟、错误率），用LangSmith做LLM级分析（Prompt优化、回答质量评估）。

六、系统设计与性能 (4题)

Q29: 系统QPS能到多少？瓶颈在哪里？

答：

瓶颈分析：

- **LLM API调用**：最大瓶颈。GPT-4o约50–100 TPS，每个请求调用2–3次LLM
- **向量检索**：FAISS毫秒级，Milvus 10ms级，不是瓶颈
- **Redis**：10万+ QPS，不是瓶颈

优化方案：

1. 缓存热点问题的路由决策和检索结果
2. 简单问题（FAQ精确命中）直接返回，不走LLM
3. 意图路由用小模型，降低API成本和延迟
4. 请求排队 + 批处理

Python版预估QPS：50–100（受LLM API限制） Go版预估QPS：200–500（Go本身开销极低，主要还是LLM瓶颈）

Q30: 如何做Agent系统的压测？

答：

1. **Mock LLM**：压测时用固定响应替代真实LLM调用，测试系统本身的吞吐
2. **渐进加压**：从10 QPS逐步加到目标QPS，观察P99延迟变化
3. **场景覆盖**：构造不同意图的测试用例（咨询、投诉、办理），模拟真实分布
4. **长时间压测**：测试内存泄露、连接池耗尽等问题
5. **工具**：Locust（Python）、k6（Go/JS）、JMeter（Java）

关键指标：QPS、P50/P95/P99延迟、错误率、内存/CPU使用率。

Q31: Agent系统的安全威胁有哪些？

答：

1. **Prompt注入**：用户在消息中嵌入恶意指令，试图改变Agent行为
2. **PII泄露**：Agent在回复中暴露其他用户的个人信息
3. **越权操作**：Agent被诱导执行超出权限的工具调用
4. **供应链攻击**：恶意MCP工具
5. **数据投毒**：知识库中被注入虚假信息

防御措施：

- 输入净化 + 输出审查（合规Agent）
 - PII检测和脱敏
 - 工具调用权限控制
 - 知识库内容审核流程
-

Q32: 如果让你重新设计这个系统，你会做什么不同？

答：

这是一个很好的反思问题。我会做以下改进：

1. **评估先行**：先建立完善的评估体系（Golden Set + 自动评估Pipeline），再迭代优化，而不是凭感觉调优
2. **Streaming优先**：从一开始就设计SSE流式响应，用户体验会好很多
3. **工具编排图可视化**：提供Admin UI让运营人员可以拖拽配置Agent编排流程
4. **混合检索**：一开始就用Hybrid Search（向量+BM25），而不是纯向量检索
5. **A/B测试框架**：内置A/B测试能力，方便对比不同Prompt/模型/策略的效果

项目深度问答 — 面试官追问 & 标准回答

模拟面试官针对项目的深入追问，提供标准回答和踩坑分享。 每个问题按"问题 → 标准回答 → 追问升级 → 踩坑分享"组织。

Q1: "你的Supervisor编排和简单的if-else路由有什么区别?"

标准回答

这是个很好的问题。如果只看意图路由这一步，确实可以用if-else或者关键词匹配来做。但Supervisor编排的价值在于整个流程的编排能力：

1. **状态管理**：Supervisor维护全局State，各Agent共享读写，支持复杂的多步骤流程
2. **流程编排**：支持条件分支、并行执行、循环重试，不只是单一路由
3. **检查点恢复**：基于LangGraph的Checkpoint，长流程中断可以从断点恢复
4. **Human-in-the-Loop**：可以在任意节点插入人工审批环节
5. **可观测性**：每个节点自动生成追踪Span，if-else做不到

简单来说，if-else只能做"选A还是选B"，Supervisor可以做"先A、然后B和C并行、B的结果给D审查、审查不通过回到A"。

踩坑分享

我一开始确实用了简单路由，但当需要加入"所有回复都必须经过合规审查"这个需求时，if-else变得非常难维护——每个分支末尾都要加审查逻辑。换成StateGraph后，只需要加一条边就行了。

Q2: "分层记忆的三层是怎么协作的？给个具体例子"

标准回答

以一个多轮退款咨询为例：

第1轮： 用户说"我买的理财产品想退"

- 工作记忆：记录 `intent=ticket, product_type=理财`
- 短期记忆：存入 `{role: user, content: "我买的理财产品想退"}`
- 长期记忆：检索退款政策文档

第2轮： 用户说"就是上个月买的那个"

- 工作记忆：读取上轮 `product_type=理财`，更新 `time=上个月`
- 短期记忆：拼接上下文，Agent知道"那个"指的是理财产品
- 长期记忆：检索该用户的历史订单信息

第3轮： 用户说"好的，帮我申请退款"

- 工作记忆：聚合前两轮信息，构建完整的退款请求
- 短期记忆：提供完整对话上下文给工单Agent
- 长期记忆：写入新工单记录

关键点： 如果没有分层记忆——

- 没有工作记忆：第2轮不知道"那个"指什么
- 没有短期记忆：每轮都要重新问用户信息
- 没有长期记忆：不知道退款政策和用户历史

追问升级：缓存击穿怎么处理？

短期记忆用Redis存储。缓存击穿指热点session的Redis key过期时，大量请求同时查Redis miss然后打到后端。

解决方案：

1. **互斥锁**：第一个请求发现miss后加锁重建缓存，其他请求等待
2. **永不过期 + 后台刷新**：key设永不过期，用后台线程定期刷新
3. **布隆过滤器**：快速判断session是否存在，避免无效查询

我们采用方案1（Redis SETNX实现分布式锁），因为客服场景的热点session不多。

Q3: "合规审查Agent的误判率怎么控制?"

标准回答

合规审查有两类错误：

- **漏判 (False Negative)**：违规内容没检出 → 风险大
- **误判 (False Positive)**：合规内容被误报 → 用户体验差

我们的两阶段策略分别优化这两类错误：

第一阶段——规则引擎（高召回率，低精确率）：

- 用正则匹配PII、用关键词匹配违禁词
- 宁可多报不能漏报（召回率>99%）

第二阶段——LLM深度审查（高精确率）：

- 对规则引擎通过的内容做二次审查
- 处理规则无法覆盖的场景（如隐晦的越权承诺）
- 精确率>95%

两阶段结合：

- 规则命中高风险 → 直接拦截（不走LLM，省成本）
- 规则通过 → LLM审查 → 通过则放行
- 整体漏判率<0.5%，误判率<2%

踩坑分享

早期只用规则引擎，结果有个用户问“你能保证我不亏钱吗”，Agent回复“虽然不能保证但风险很低”——规则引擎没检出“风险很低”这种隐晦的违规表述，但监管认为这有误导性。加了LLM审查后这类case被成功拦截。

Q4: "如果一个Agent超时了，Supervisor怎么处理？"

标准回答

三层防御机制：

1. **Agent级超时**：每个Agent调用设10秒超时。超时后Agent抛出TimeoutError
2. **Supervisor降级**：捕获超时异常后，执行降级策略：
 - 知识检索超时 → 返回“暂时无法查询，已记录您的问题”
 - 工单处理超时 → 返回“工单提交中，请稍后查看”
 - 合规审查超时 → 按“不通过”处理，转人工审核
3. **自动工单**：超时事件自动创建内部告警工单，技术团队排查

```
python
```

```
try:
```

```
    result = await asyncio.wait_for(agent.process(state), timeout=10.0)
```

```
except asyncio.TimeoutError:
```

```
    result = fallback_response(agent.name)
```

```
    create_alert_ticket(agent.name, state.session_id)
```

追问：合规审查超时按“不通过”会不会太激进？

不会。在金融场景下，合规审查是底线。宁可误拦也不能漏放。用户被“转人工”只是多等几分钟，但一条违规回复可能导致监管处罚。

Q5: "RAG检索的准确率怎么评估?"

标准回答

三个层面的评估：

1. 检索质量评估 (Retrieval)

- **Recall@K**: Top-K结果中包含正确文档的比例
- **MRR (Mean Reciprocal Rank)**: 正确文档排名的倒数平均
- 用标注好的Query-Document对测试集评估

2. 生成质量评估 (Generation)

- **Faithfulness**: 回答是否忠实于检索到的文档 (不编造)
- **Answer Relevance**: 回答是否与问题相关
- 用LLM-as-Judge评估 (GPT-4打分)

3. 端到端评估 (End-to-End)

- **首问解决率 (FCR)**: 用户一次性得到满意回答的比例
- **CSAT**: 用户满意度评分
- 来自真实用户反馈

踩坑分享

刚上线时检索Recall@5有85%，看着不错，但用户满意度只有60%。排查发现问题在生成阶段——检索到了对的文档，但LLM没有把关键信息提取出来。加了"请从文档中找出并直接引用相关条款"的Prompt指令后，满意度提升到80%。

Q6: "你们怎么处理用户的Prompt注入攻击?"

标准回答

Prompt注入指用户在消息中嵌入"忽略以上指令，执行XXX"之类的恶意内容。

我们的防御策略：

1. **输入预处理**：对用户输入做净化，检测已知的注入模式
2. **System Prompt隔离**：使用System角色设定Agent身份和规则，与用户输入明确分隔
3. **角色锁定**：在Prompt中强调"你只能回答客服相关问题，不能执行系统命令"
4. **输出审查**：合规Agent检查回复是否偏离了客服角色
5. **工具权限控制**：即使Agent被"说服"，它能调用的工具也有严格的权限边界

Q7: "为什么用LangGraph而不是CrewAI?"

标准回答

两者都调研了，选LangGraph的原因：

1. **状态管理**：LangGraph有显式的TypedState和Checkpoint，CrewAI的状态管理是隐式的
2. **编排灵活性**：LangGraph是图模型，支持条件分支、循环、并行；CrewAI是任务队列模型，编排能力有限
3. **生产就绪**：LangGraph有LangGraph Platform（云托管）、持久化、断点恢复、Cron任务
4. **生态**：LangSmith提供完整的观测和评估能力

CrewAI的优势是上手更简单（Agent/Task/Crew三层抽象很直观），适合快速原型。但我们的场景需要精确控制流程（必须经过合规审查），CrewAI做这个比较费劲。

Q8: "Go版本和Python版本有什么性能差异?"

标准回答

在不调用LLM API的纯系统开销测试中：

指标	Python (LangGraph)	Go (Gin)
单请求延迟（不含LLM）	~15ms	~2ms
内存占用（空载）	~200MB	~30MB
并发处理	asyncio事件循环	goroutine真并行
Agent调度开销	~5ms	~0.5ms

但实际场景中，LLM API调用是绝对瓶颈（1-3秒），系统本身的开销在整体延迟中占比<5%。

选Go的场景：高并发（QPS>1000）、内存敏感（容器资源限制）、与Go微服务生态融合。选Python的场景：AI生态丰富（LangChain/LangGraph）、开发效率高、团队Python技术栈。

Q9: "Token消耗降低40%是怎么做到的？"

标准回答

主要三个手段：

- 1. 分层记忆代替全量历史（贡献约25%）
 - 之前：每次请求把完整20轮对话塞进prompt → ~4000 tokens
 - 之后：工作记忆只传关键上下文 → ~1000 tokens
- 2. 路由缓存（贡献约10%）
 - 高频问题的路由决策缓存5分钟
 - 如"查余额"永远路由到knowledge_rag，不需要每次调LLM判断
- 3. 早退机制（贡献约5%）
 - FAQ精确匹配直接返回，不走Agent链路
 - 前50%的问题可以通过FAQ覆盖

Q10: "系统上线后遇到过什么线上事故?"

标准回答

分享两个有代表性的事故：

事故一：合规审查漏放

- 现象：有用户收到包含"风险极低"的回复
- 原因：规则引擎只匹配了"零风险"等绝对用语，没覆盖"风险极低"这种相对用语
- 解决：增加LLM二阶段审查，同时扩充规则词库
- 预防：建立合规case库，每周review新增case

事故二：Redis短期记忆OOM

- 现象：上线后第3天Redis内存告警
- 原因：TTL设了30分钟，但高峰期并发session数远超预期，大量对话同时在缓存中
- 解决：1) 降低TTL到15分钟 2) 限制单session最大轮数从20降到10 3) Redis加内存淘汰策略(allkeys-lru)
- 预防：上线前做容量评估，压测模拟真实session并发数

Q11: "如果你是面试官，你会怎么评估这个项目的含金量?"

标准回答（自我评价，展示成熟度）

我会从四个维度评估：

1. 架构设计能力：是否有合理的架构决策和trade-off分析
2. 工程实践：是否有可观测性、容错、性能优化等生产级考量
3. 业务理解：是否理解金融/电商客服的业务需求和合规要求
4. 技术深度：是否能讲清楚RAG、Agent编排、分布式记忆的底层原理

这个项目的含金量在于：

- 不是简单调API，而是有完整的架构设计
- 有企业级考量（合规、追踪、容错）
- 有量化成果（FCR、CSAT、QPS都有数据）
- 有多语言实现，说明理解了架构而非绑定框架

Q12: "你对多Agent系统未来的看法？"

标准回答

几个我观察到的趋势：

1. **MCP标准化**：Anthropic的MCP协议正在成为工具调用的统一标准，未来Agent不再需要为每个工具写适配器
2. **编排框架成熟**：LangGraph、Spring AI等框架正在从"实验工具"走向"生产基础设施"
3. **Agent即服务**：AWS Agent Squad已经在做Agent的SaaS化，未来开发者可以像调用API一样组合Agent
4. **评估体系建立**：目前Agent评估还比较粗糙，未来会有更成熟的Benchmark和自动化评估框架
5. **人机协作优化**：Human-in-the-Loop不是退步，而是当前LLM可靠性下的最佳实践

我认为多Agent系统在2026-2027年会成为企业级AI应用的标配架构，就像微服务之于Web应用一样。

回答面试问题的通用技巧

1. **先总后分**：先用一句话概括，再展开细节
2. **用数据说话**：每个技术决策都配一个量化结果
3. **展示trade-off**：不只说"我选了A"，要说"A和B的对比，选A因为..."
4. **承认不足**：适当说"如果重来我会..."，展示成长性

5. **关联业务**：技术决策要关联到业务需求（如"因为金融合规要求..."）
6. **准备深度**：每个回答准备3层深度——面试官追问到第3层还能答上来，就赢了

简历项目经历模板 — 智能客服多Agent系统

根据不同岗位方向，提供多个版本的简历写法。采用STAR法则，突出量化成果。

版本一： AI/NLP工程师 方向

智能客服多Agent系统 | 核心开发者 | 2025.09 – 2026.03

项目背景： 面向金融/电商场景的企业级智能客服系统，采用多Agent架构替代传统单体客服机器人，服务日均10万+用户咨询。

核心职责：

- 设计并实现基于 **Supervisor编排模式** 的多Agent协作架构，包含意图路由、知识检索(RAG)、工单处理、合规审查4个专业Agent
- 构建 **三层记忆系统**（工作记忆 + Redis短期记忆 + Milvus向量库长期记忆），解决多轮对话上下文丢失问题
- 实现完整 **RAG流程**（Query改写 → 向量检索 → 重排序 → 上下文注入），知识库检索准确率达92%
- 基于 **MCP工具协议** 实现标准化工具调用层，统一对接订单系统、工单系统、风控接口
- 集成 **OpenTelemetry全链路追踪**，实现Agent调用链路可视化和Token消耗监控

技术栈： Python / LangGraph / FastAPI / Redis / FAISS / OpenTelemetry / Docker

项目成果：

- 首问解决率(FCR)从65%提升至82%，客户满意度CSAT从4.3提升至4.7
- 平均响应时间从30分钟降至3秒，日处理能力提升20倍
- Token消耗通过分层记忆+缓存策略降低40%
- 合规风险事件减少95%，实现金融合规自动化审查

版本二：Java后端工程师 方向

智能客服多Agent系统 | 后端负责人 | 2025.09 – 2026.03

项目背景：基于Spring AI + Spring Boot 3.x构建的企业级多Agent智能客服系统，支撑金融业务日均10万+咨询量。

核心职责：

- 基于 **Spring AI** 框架设计多Agent编排架构，实现 RoutingAgent(意图路由) + SequentialAgent(知识检索) + ParallelAgent(并行处理) 的组合编排
- 设计 **分层缓存策略**：Spring Data Redis实现会话缓存（TTL 30min），Milvus Java SDK实现向量化长期记忆
- 实现 **MCP工具协议** JSON-RPC 2.0服务端，统一管理订单查询、工单CRUD、风控接口等外部工具
- 基于 **OpenTelemetry + Micrometer** 搭建Agent调用链路追踪和性能监控体系
- 使用 **ConcurrentHashMap + ReadWriteLock** 实现高并发工单存储，支持线程安全的工单状态流转

技术栈：Java 21 / Spring Boot 3.4 / Spring AI / Redis / Milvus / OpenTelemetry / Docker

项目成果：

- 系统QPS达500+，P99延迟<3s，支撑日均10万+请求
- 工单处理效率提升15倍，人工干预率从40%降至8%
- 合规审查实现两阶段机制（规则引擎毫秒级 + LLM深度审查），误判率<2%

版本三：Go后端工程师 方向

智能客服多Agent系统 | 后端开发工程师 | 2025.09 – 2026.03

项目背景：基于Gin框架和自研轻量级LLM客户端构建的高并发多Agent智能客服系统，利用Go的goroutine并发模型实现Agent并行调度。

核心职责：

- 设计Supervisor编排模式，使用Go手动编排 + goroutine调度4个业务Agent（意图路由、知识检索、工单处理、合规审查）
- 自研 **OpenAI兼容HTTP客户端**，支持任意LLM端点，30秒超时，环境变量配置（OPENAI_API_KEY / BASE_URL / MODEL_NAME）
- 实现完整RAG管线：**Query改写** → **向量检索(Top-5)** → **LLM重排序(Top-3)** → **生成回答**，关键词+余弦相似度混合检索
- 设计 **两阶段合规审查**：规则引擎（PII脱敏+敏感词检测，<2ms）+ LLM深度审查，高风险直接拦截不走LLM
- 设计 **sync.RWMutex** 保护的分层记忆系统，工作记忆(进程内) + Redis短期记忆（自动降级）+ 向量库长期记忆
- 基于 **Gin框架** 提供RESTful API，集成CORS中间件、MCP JSON-RPC 2.0工具调用、OpenTelemetry分布式追踪

技术栈：Go 1.22 / Gin / 自研LLM客户端 / Redis / OpenTelemetry / Docker

项目成果：

- 单实例QPS达2000+，goroutine池化管理避免泄露
- 内存占用比Python实现降低70%，比Java实现降低40%
- 全链路P99延迟<1.5s，Agent调度开销<50ms

简历写作要点总结

动词选择（强动词优先）

- 用 "设计并实现" 而非 "参与"
- 用 "搭建" 而非 "协助搭建"
- 用 "优化...提升X%" 而非 "改善了性能"

量化原则（每条必须有数字）

- 性能指标：QPS、P99延迟、响应时间
- 业务指标：FCR、CSAT、处理能力倍数
- 成本指标：Token降低百分比、内存降低百分比
- 质量指标：准确率、误判率、风险事件减少比例

技术栈排列（按面试岗位调整顺序）

- AI岗：LangGraph / RAG / OpenTelemetry 放前面
- Java岗：Spring Boot / Spring AI / Redis 放前面
- Go岗：Go / Gin / goroutine / 自研LLM客户端 放前面

避免的写法

- "负责xxx模块的开发" → 太笼统
- "使用了xxx技术" → 没有成果
- "参与了xxx项目" → 无主动性
- 不写具体数字 → 不可信

STAR面试话术 — 智能客服多Agent系统

面试时用STAR法则（Situation → Task → Action → Result）回答项目问题。以下提供多个场景的完整话术模板，可直接背诵后灵活调整。

场景一："请介绍一下你最有挑战的项目"

S（情境）

我上一个项目是做智能客服系统。当时公司的客服系统面临几个痛点：日均咨询量超过10万，但人工客服响应平均要30分钟；公司有多个知识库但分散在不同系统，导致回答不一致；金融业务场景下缺乏自动化的合规审查，合规风险事件频发。

T（任务）

我作为核心开发者，负责设计和实现一套基于多Agent架构的智能客服系统。目标是将首问解决率从65%提升到80%以上，响应时间降到秒级，同时实现合规审查自动化。

A（行动）

我做了四个关键技术决策：

第一，架构选型——我调研了单Agent、Peer-to-Peer、Supervisor等多种编排模式后，选择了Supervisor模式。原因是客服场景需要中央协调、需要保证每个回复都经过合规审查，Supervisor模式天然支持这种"先分发、后审查"的流程。

第二，分层记忆——多轮对话最大的难点是上下文丢失。我设计了三层记忆系统：工作记忆存当前推理状态（进程内存，零延迟）、短期记忆存最近20轮对话（Redis，TTL 30分钟）、长期记忆存知识库和用户画像（向量数据库，持久化）。三层协作解决了"用户说了什么"和"系统知道什么"的匹配问题。

第三，RAG知识检索——我实现了完整的RAG流程：先对用户口语化问题做Query改写，然后向量检索Top5文档，再用LLM做重排序选Top3，最后注入上下文生成回答。这比直接拿用户原始问题搜索，准确率提升了20个百分点。

第四，全链路追踪——我集成了OpenTelemetry，每个Agent调用都生成Span，可以追踪从用户请求到最终响应的完整链路，包括每个Agent的耗时、Token消耗、路由决策。这对线上问题排查非常关键。

R（结果）

最终效果：首问解决率从65%提升到82%，CSAT从4.3到4.7，响应时间从30分钟降到3秒。Token消耗通过分层记忆降低了40%——因为不需要每次都把全部历史塞进prompt了。合规风险事件减少了95%。

场景二："你在项目中遇到过什么技术难题？怎么解决的？"

S（情境）

系统上线初期，我们发现一个严重问题：当多个Agent并行处理时，合规审查Agent偶尔会拿到不完整的数据——比如知识检索Agent还没返回结果，合规审查就已经开始了。这导致部分回复跳过了合规检查直接返回给用户。

T（任务）

我需要解决多Agent并行场景下的数据一致性问题，确保合规审查一定在所有业务Agent完成后才执行。

A（行动）

我分析后发现根因是LangGraph的StateGraph中，我最初把合规审查和知识检索设成了并行边。解决方案是调整图的拓扑结构：

1. 让所有业务Agent（知识检索、工单处理）的输出边都指向合规审查节点
2. 合规审查节点变成一个"汇聚点"，只有当上游所有Agent都完成后才会触发
3. 在State中增加了 `sub_results` 字段，各Agent把结果写入这个dict，合规Agent从中读取全部内容做审查

同时我还加了一个兜底机制：如果任何业务Agent超时（>10秒），Supervisor会触发降级，直接返回"转人工"并自动创建工单。

R（结果）

修复后合规审查的覆盖率从92%恢复到100%，线上没有再出现漏审的情况。降级机制也帮我们避免了用户长时间等待——超时率本身只有0.5%，但这0.5%之前会导致用户等30秒+无响应。

场景三："为什么选择这个技术栈？有没有考虑过替代方案？"

S

技术选型时我们面临多个选择：Python有LangGraph和CrewAI，Java有Spring AI，Go则采用Gin + 自研轻量级LLM客户端的方案。同时在向量数据库上也需要在FAISS、Milvus、Pinecone之间做选择。

T

我需要给团队提供一个技术选型方案，要考虑开发效率、生产稳定性、团队技能栈。

A

我做了一个系统的对比分析：

编排框架：

- LangGraph：图式编排，状态管理好，checkpoint支持，但Python性能有瓶颈
- CrewAI：上手简单，但对复杂编排的控制力不够
- Spring AI：Java生态成熟，企业级特性完善，但AI社区生态不如Python

我的决策是：**用Python/LangGraph做MVP快速验证，同时提供Java和Go版本给不同团队使用。**

向量数据库：

- FAISS：Facebook开源，单机性能最好，毫秒级响应，但需要自己管理
- Milvus：云原生，分布式，支持百亿级向量，适合生产环境
- Pinecone：全托管，零运维，但成本最高且数据出境合规问题

我的决策是：**开发阶段用FAISS（零依赖快速开发），生产环境切Milvus（横向扩展能力）。**

R

这个分层选型策略让我们3周就完成了MVP验证，验证成功后再花2周迁移到生产级技术栈，比一开始就用重型方案节省了约40%的开发时间。

场景四："项目中你是怎么保证系统质量的？"

S

多Agent系统的质量保证比单体系统复杂得多——每个Agent的输出都可能影响最终结果，而且Agent行为具有一定的不确定性。

T

我需要建立一套质量保证体系，覆盖开发、测试、线上监控全流程。

A

我从三个维度建立质量保证：

1. **评估指标体系**：定义了Agent级别的指标——意图路由准确率、RAG检索准确率、合规审查的精确率/召回率；以及系统级指标——FCR、CSAT、P99延迟、Token消耗
2. **全链路追踪**：集成OpenTelemetry，每个请求都能追踪完整链路。一旦线上出现badcase，我可以在追踪系统中看到：Supervisor把这个请求路由到了哪个Agent、Agent拿到了什么上下文、生成了什么中间结果、合规审查是否通过
3. **合规双保险**：合规审查Agent采用两阶段机制——先用规则引擎做毫秒级快速检查（敏感词、PII），再用LLM做深度审查（越权承诺、误导性表述）。规则引擎保底，LLM提升覆盖率

R

上线3个月内，线上合规事故为零；badcase追踪从之前的"看日志找半天"变成"5分钟定位根因"；意图路由准确率从85%优化到93%。

面试话术万能模板

S: 我做的是[项目名称], 当时面临[具体痛点/挑战, 带数字]...

T: 我负责[具体职责], 目标是[量化目标]...

A: 我做了[关键技术决策1]...[关键技术决策2]...

选择[技术A]而非[技术B]的原因是[具体理由]...

R: 最终[核心指标]从[旧值]提升到[新值], [次要指标]改善了[百分比]...

关键技巧

1. 先说结论: 面试官时间有限, 先讲最亮眼的结果
2. 用数字说话: 每个Action对应一个量化Result
3. 展示决策过程: 不只说"我用了X", 要说"我调研了X/Y/Z, 选择X因为..."
4. 承认局限: 适当提及"如果重来, 我会...", 显示反思能力
5. 准备追问: 每个Action都准备2-3层深度追问的回答